

01-monitoring-vs-observability

Monitoring vs Observability

1. Overview
 2. What is Monitoring?
 - Characteristics of good monitoring
 3. What is Observability?
 - Why Observability Matters More in Distributed Systems
 4. Key Relationship
 5. Practical Example
 6. The Three Pillars in Detail
 - Metrics
 - Logs
 - Traces
 7. Common Tooling Stack
 8. Summary Takeaways
- Lab: Comparing Monitoring vs Observability Hands-On
- Prerequisites
 - Step 1 — Spin up a simple monitored app
 - Step 2 — Simulate load and observe via Monitoring only
 - Step 3 — Add tracing to gain Observability
 - Step 4 — Reflection questions
 - Cleanup

Monitoring vs Observability

1. Overview

Monitoring and **Observability** are related but distinct concepts in modern system operations. Understanding the difference helps teams design better systems for detecting, diagnosing, and resolving issues.

Aspect	Monitoring	Observability
Definition	Watching predefined metrics/logs against known thresholds	Ability to understand a system's internal state from its external outputs
Focus	"Is something wrong?"	"Why is something wrong?"
Approach	Reactive — alerts on known failure modes	Proactive/exploratory — investigate unknown-unknowns
Data used	Dashboards, alerts, predefined metrics	Metrics + Logs + Traces (the "three pillars"), high-cardinality data
Best for	Known failure patterns (CPU, memory, latency thresholds)	Novel, complex, or previously unseen failure patterns

2. What is Monitoring?

Monitoring is the practice of collecting, aggregating, and analyzing metrics and logs over time to track the health and performance of a system. It typically answers questions you already know to ask:

- Is the CPU usage above 80%?
- Is the error rate above 1%?
- Is the service responding within 200ms?

Monitoring systems are built around **dashboards** and **alerts** tied to known thresholds (SLOs/SLIs). Tools: Prometheus, Nagios, Zabbix, Datadog, CloudWatch.

Characteristics of good monitoring

- Clear, actionable alerts (avoid alert fatigue)
- Well-defined SLIs/SLOs (Service Level Indicators/Objectives)
- Dashboards for at-a-glance health checks
- Historical trend analysis

3. What is Observability?

Observability is a property of a system — how well its internal state can be inferred from the data it emits (telemetry). It comes from control theory: a system is “observable” if you can determine its internal state purely from its outputs.

In software systems, observability is built on three pillars:

1. **Metrics** — numeric measurements over time (counters, gauges, histograms)
2. **Logs** — timestamped, discrete event records
3. **Traces** — the path of a single request across distributed services

A fourth pillar increasingly discussed is **Events/Profiles** (continuous profiling), used to correlate CPU/memory hotspots with specific code paths.

Why Observability Matters More in Distributed Systems

In monolithic systems, you often know where to look when something breaks. In microservices/distributed/cloud-native systems:

- Failures can cascade across dozens of services
- You cannot predict every failure mode in advance
- Root cause analysis requires correlating metrics, logs, and traces together

Observability enables engineers to ask **arbitrary new questions** about system behavior without shipping new code — this is the key differentiator from monitoring, which only answers questions defined in advance.

4. Key Relationship

Monitoring tells you **that** something is wrong. Observability helps you understand **why** it's wrong.

Monitoring is a subset of the observability toolkit — you need monitoring (metrics + alerting) *and* deep introspection (logs, traces, high-cardinality queries) to have full observability.

5. Practical Example

Scenario: An e-commerce checkout API's p99 latency spikes.

- **Monitoring** tells you: “p99 latency > 2000ms since 14:32 UTC” (alert fires).
- **Observability** lets you: drill into a distributed trace of a slow request, see it spent 1.8s waiting on a downstream payment-service call, check that service's logs, and discover a specific database connection pool was exhausted only for requests routed to a particular pod — something you never explicitly instrumented an alert for.

6. The Three Pillars in Detail

Metrics

- Cheap to store, good for trends and alerting
- Low cardinality by default (labels can raise cardinality — costly in Prometheus)
- Example: `http_requests_total`, `cpu_usage_percent`

Logs

- Rich context, but expensive to store/search at scale
- Structured logging (JSON) is preferred over plain text for queryability
- Example: `{"level": "error", "service": "checkout", "trace_id": "abc123", "msg": "payment timeout"}`

Traces

- Show causality and latency breakdown across service boundaries
- Standard: OpenTelemetry (OTel), Jaeger, Zipkin
- Each trace is composed of **spans** representing units of work

7. Common Tooling Stack

Pillar	Common Tools
Metrics	Prometheus, Grafana, Cortex, Thanos
Logs	Loki, ELK (Elasticsearch/Logstash/Kibana), Splunk
Traces	Jaeger, Zipkin, Tempo, OpenTelemetry Collector
Unified platforms	Datadog, New Relic, Honeycomb, Grafana Cloud

8. Summary Takeaways

- Monitoring = watching known signals against known thresholds
- Observability = the capability to explore and understand unknown failure modes
- You need both: monitoring for fast detection, observability for deep diagnosis
- The three pillars (metrics, logs, traces) are the building blocks of observability
- Investing in structured logs, trace context propagation, and good metric labeling early pays off enormously during incidents

Lab: Comparing Monitoring vs Observability Hands-On

Goal: Experience the practical difference by simulating an incident with only metrics, then again with full observability data.

Prerequisites

- Docker & Docker Compose installed
- Basic familiarity with the command line

Step 1 — Spin up a simple monitored app

```

mkdir mon-vs-obs-lab && cd mon-vs-obs-lab
cat > docker-compose.yml <<'EOF'
version: "3.8"
services:
  app:
    image: kennethreitz/httpbin
    ports:
      - "8080:80"
  prometheus:
    image: prom/prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
EOF

cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: 'app'
    static_configs:
      - targets: ['app:80']
EOF

docker compose up -d

```

Step 2 — Simulate load and observe via Monitoring only

```
for i in $(seq 1 200); do curl -s http://localhost:8080/delay/1 > /dev/null; done
```

Open `http://localhost:9090` and query:

```
rate(prometheus_http_request_duration_seconds_count[1m])
```

Notice: you can see request rate/latency trending, but **no insight into which specific request or code path is slow.**

Step 3 — Add tracing to gain Observability

Add Jaeger to `docker-compose.yml`:

```

jaeger:
  image: jaegertracing/all-in-one
  ports:
    - "16686:16686"
    - "4318:4318"

```

Re-run `docker compose up -d`, then instrument a small test script (e.g., using OpenTelemetry Python SDK) to send a traced request to `/delay/1`.

Open Jaeger UI at `http://localhost:16686` and inspect the trace — you can now see **exactly where the 1s delay occurred** in the call chain, something the metrics-only view could not show.

Step 4 — Reflection questions

1. What could the Prometheus dashboard tell you that the trace couldn't?
2. What could the trace tell you that Prometheus couldn't?
3. How would structured logs with a `trace_id` field help you pivot between these two views during a real incident?

Cleanup

```
docker compose down -v
```